

Relazione — Gestione dei Permessi in Linux

Cyber Security & Ethical Hacking — Esercizio | Francesco Terracciano

Obiettivo

Configurare e testare i permessi di lettura, scrittura ed esecuzione su un file in ambiente Linux (Kali Linux), utilizzando i comandi **chmod** e **ls -l**.

Passaggi eseguiti

1. È stata creata una directory di lavoro **test_permessi** e al suo interno un file **script_test.sh** tramite il comando **touch**.
2. Verificando i permessi iniziali con **ls -l**, il file risultava con permessi **-rw-rw-r--** (664): lettura e scrittura per proprietario e gruppo, sola lettura per gli altri.

Motivazione delle scelte sui permessi

I permessi sono stati modificati con **chmod 750**, ottenendo **-rwxr-x---**:

- **Proprietario (rwx)**: lettura, scrittura ed esecuzione complete, in quanto è l'utente che deve poter modificare ed eseguire lo script liberamente.
- **Gruppo (r-x)**: lettura ed esecuzione ma non scrittura, per permettere ad altri membri fidati del gruppo di usare lo script senza poterlo alterare.
- **Altri (---)**: nessun permesso, per impedire a utenti non autorizzati di leggere, modificare o eseguire il file, dato che potrebbe contenere informazioni sensibili o operazioni critiche.

Successivamente, per testare uno scenario più restrittivo, è stato impostato **chmod 444 (-r--r--r--)**, rimuovendo ogni permesso di scrittura ed esecuzione per tutti gli utenti, incluso il proprietario.

Analisi dei risultati dei test

Con permessi **750**, il tentativo di scrittura sul file (**echo "test scrittura" >> script_test.sh**) è andato a buon fine, poiché eseguito come proprietario del file (**whoami** → **kali**), che dispone del permesso di scrittura.

Con permessi **444**, il medesimo tentativo di scrittura (**echo "prova" >> script_test.sh**) è stato rifiutato dal sistema con l'errore **permission denied**, confermando che la rimozione del permesso di scrittura (w) blocca correttamente l'operazione anche per il proprietario del file.

Questo test dimostra che il modello di permessi Linux (owner/group/other, rwx) viene applicato in modo granulare e coerente: la revoca del bit di scrittura impedisce l'operazione indipendentemente da chi la esegue, comprovando l'efficacia del meccanismo di controllo degli accessi.

Allegati — Screenshot dei passaggi

Figura 1

```
(kali㉿kali)-[~]  
$ mkdir test_permessi  
cd test_permessi  
touch script_test.sh  
  
(kali㉿kali)-[~/test_permessi]  
$ ls -l script_test.sh  
-rw-rw-r-- 1 kali kali 0 Aug 25 08:30 script_test.sh
```

Screenshot 1 — Creazione della directory e del file, verifica permessi iniziali (ls -l).

Figura 2

```
(kali㉿kali)-[~/test_permessi]  
$ chmod 750 script_test.sh  
ls -l script_test.sh  
-rwxr-x--- 1 kali kali 0 Aug 25 08:30 script_test.sh  
  
(kali㉿kali)-[~/test_permessi]
```

Screenshot 2 — Modifica dei permessi con chmod 750 e verifica con ls -l.

Figura 3

```
(kali㉿kali)-[~/test_permessi]  
$ echo "test scrittura" >> script_test.sh  
mia_regola...  
(kali㉿kali)-[~/test_permessi]  
$ whoami  
kali
```

Screenshot 3 — Test di scrittura riuscito come utente proprietario (whoami: kali).

Figura 4

```
(kali㉿kali)-[~/test_permessi]  
$ chmod 444 script_test.sh  
echo "prova" >> script_test.sh  
zsh: permission denied: script_test.sh
```

Screenshot 4 — chmod 444 e tentativo di scrittura bloccato (permission denied).